



Bikkad IT Institute

Session 9: Inheritance & Polymorphism (OOP-3)

Compiled by Bikkad IT Institute | Teaching Notes

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Python Programming - Beginner Level Notes

Prepared for: Bikkad IT Institute Students

Topic: OOP Part 3 - Advanced Concepts

Date: February 2026

1. Class Relationships

Definition

Class relationships define how classes interact with each other.

Types of Relationships:

- Association - "uses a" relationship
- Aggregation - "has a" relationship (weak)
- Composition - "owns a" relationship (strong)
- Inheritance - "is a" relationship

2. Aggregation (Has-A Relationship)

What is Aggregation?

A relationship where one class **contains** another class as a member, but both can exist independently.

Real-World Examples:

- Car has Engine (Engine can exist without Car)
- Department has Employees (Employees can exist without Department)
- Library has Books (Books can exist without Library)

Code Examples

Example 1: Basic Aggregation

```
class Engine:
```

```
def init(self, horsepower):
```

```
self.horsepower = horsepower
```

```
def start(self):
```

```
    print(f"Engine with {self.horsepower}HP started")
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

```
class Car:
    def init(self, brand, engine):
        self.brand = brand
        self.engine = engine # Has-A relationship
```

```
    def start_car(self):
        print(f"{self.brand} starting...")
        self.engine.start()
```

Engine exists independently

```
engine1 = Engine(150)
```

Car has Engine

```
car1 = Car("Maruti", engine1)
car1.start_car()
```

Example 2: Department and Employee

```
class Employee:
    def init(self, name, emp_id):
        self.name = name
        self.emp_id = emp_id
```

```
    def display(self):
        print(f"{self.name} - {self.emp_id}")
```

```
class Department:
    def init(self, dept_name):
        self.dept_name = dept_name
        self.employees = [] # List of employees
```

```
    def add_employee(self, employee):
        self.employees.append(employee)

    def show_employees(self):
        print(f"Department: {self.dept_name}")
        for emp in self.employees:
            emp.display()
```

OUR LOCATIONS

Employees exist independently

```
emp1 = Employee("Rahul", 101)
emp2 = Employee("Priya", 102)
```

Department has Employees

```
it_dept = Department("IT")
it_dept.add_employee(emp1)
it_dept.add_employee(emp2)
it_dept.show_employees()
```

3. Inheritance

What is Inheritance?

A mechanism where a new class (child) derives properties and behaviors from an existing class (parent).

Benefits:

- Code reusability
- Establishes relationship between classes
- Reduces code duplication
- Easy to maintain and extend

Syntax:

```
class Parent:
# parent code
```

```
class Child(Parent):
# child code
```

Code Examples

Example 1: Basic Inheritance

Parent class

```
class Animal:
def init(self, name):
    self.name = name
```

```
def eat(self):
    print(f"{self.name} is eating")
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Child class

```
class Dog(Animal):
def bark(self):
print(f"{self.name} is barking")
```

Create child object

```
dog1 = Dog("Tommy")
dog1.eat() # Inherited from Animal
dog1.bark() # Own method
```

4. What Gets Inherited?

Inherited	Not Inherited
Public attributes	Constructor (by default)
Public methods	Private attributes (___)
Protected attributes (___)	
Class variables	

Table 1: Inheritance Rules

Code Examples

Example: What Gets Inherited

```
class Parent:
class_var = "Shared" # Inherited
```

```
def __init__(self):
    self.public = "Public"    # Inherited
    self._protected = "Protected" # Inherited
    self.__private = "Private"  # NOT directly inherited

def public_method(self): # Inherited
    print("Public method")

def __private_method(self): # NOT inherited
    print("Private method")
```

```
class Child(Parent):
pass
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

```
child = Child()
print(child.public) # Works
print(child._protected) # Works
```

print(child.__private) # Error

```
child.public_method() # Works
```

child.__private_method() # Error

5. Constructor in Inheritance

Key Rules:

- Parent constructor NOT automatically called if child has constructor
- Parent constructor IS called if child has NO constructor
- Use super() to explicitly call parent constructor

6. Ex-1: Accessing Parent Private Attribute with Getter Method

Code Example

```
class Parent:
    def init(self):
        self.__private_data = "Secret Data"
```

```
# Getter method to access private attribute
def get_private_data(self):
    return self.__private_data
```

```
class Child(Parent):
    def init(self):
        super().init() # Call parent constructor
```

```
def display_data(self):
    # Access via getter method
    print(f"Data: {self.get_private_data()}")
```

```
child = Child()
child.display_data() # Data: Secret Data
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

7. Ex-2: If Child Has Constructor, Parent Constructor NOT Called

Code Example

```
class Parent:
    def init(self):
        self.parent_data = "Parent Data"
        print("Parent constructor called")

class Child(Parent):
    def init(self):
        # Parent constructor NOT called automatically
        self.child_data = "Child Data"
        print("Child constructor called")

child = Child()
```

Output: Child constructor called (only)

This will cause error - parent_data not initialized

print(child.parent_data) # AttributeError

Solution: Use super()

```
class Child(Parent):
    def init(self):
        super().init() # Explicitly call parent constructor
        self.child_data = "Child Data"
        print("Child constructor called")

child = Child()
```

Output: Parent constructor called

Child constructor called

print(child.parent_data) # Works now

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

8. Ex-3: If Child Don't Have Constructor, Parent Constructor IS Called

Code Example

```
class Parent:
    def __init__(self):
        self.parent_data = "Parent Data"
        print("Parent constructor called")

class Child(Parent):
    # No constructor in child
    def display(self):
        print(f>Data: {self.parent_data}")

child = Child()
```

Output: Parent constructor called (automatically)

```
child.display() # Data: Parent Data
```

9. Method Overriding

What is Method Overriding?

Child class provides its own implementation of a method that exists in parent class.

Code Examples

Example 1: Basic Overriding

```
class Animal:
    def sound(self):
        print("Animal makes sound")

class Dog(Animal):
    def sound(self): # Override parent method
        print("Dog barks")

class Cat(Animal):
    def sound(self): # Override parent method
        print("Cat meows")

dog = Dog()
dog.sound() # Dog barks

cat = Cat()
cat.sound() # Cat meows
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Example 2: Overriding with super()

```
class Employee:
    def init(self, name, salary):
        self.name = name
        self.salary = salary
```

```
    def display(self):
        print(f"Name: {self.name}")
        print(f"Salary: {self.salary}")
```

```
class Manager(Employee):
    def init(self, name, salary, department):
        super().init(name, salary)
        self.department = department
```

```
    def display(self): # Override
        super().display() # Call parent method
        print(f"Department: {self.department}")
```

```
mgr = Manager("Rahul", 50000, "IT")
mgr.display()
```

10. Super Keyword

What is super()?

super() is used to call methods from parent class.

Uses:

- Call parent constructor
- Call parent methods
- Access parent attributes

Code Examples

Example 1: super() in Methods

```
class Parent:
    def show(self):
        print("Parent show method")

class Child(Parent):
    def show(self):
        super().show() # Call parent method
        print("Child show method")
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

```
child = Child()
child.show()
```

Output: Parent show method

Child show method

Example 2: super() with Multiple Levels

```
class GrandParent:
    def display(self):
        print("GrandParent")
```

```
class Parent(GrandParent):
    def display(self):
        super().display()
        print("Parent")
```

```
class Child(Parent):
    def display(self):
        super().display()
        print("Child")
```

```
child = Child()
child.display()
```

Output: GrandParent

Parent

Child

11. Super Constructor

Using super() in Constructor

Example 1: Basic Super Constructor

```
class Vehicle:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

class Car(Vehicle):
    def __init__(self, brand, model, fuel_type):
        super().__init__(brand, model) # Call parent constructor
        self.fuel_type = fuel_type
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

```
car = Car("Maruti", "Swift", "Petrol")
print(car.brand) # Maruti
print(car.model) # Swift
print(car.fuel_type) # Petrol
```

Example 2: Multiple Inheritance super()

```
class A:
    def __init__(self):
        print("A init")
        self.a = "A"

class B:
    def __init__(self):
        print("B init")
        self.b = "B"

class C(A, B):
    def __init__(self):
        super().__init__() # Calls A's init (MRO order)
        self.c = "C"

c = C()
```

Output: A init

12. Using Super Outside Class

Cannot Use super() Outside Class Context

Example:

```
class Parent:
    def show(self):
        print("Parent method")

class Child(Parent):
    def display(self):
        super().show() # OK - inside class

child = Child()
child.display() # Works
```

super().show() # ERROR - outside class context

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

13. Types of Inheritance

1. Single Inheritance

One child class inherits from one parent class.

```
class Parent:
def parent_method(self):
print("Parent method")
```

```
class Child(Parent):
def child_method(self):
print("Child method")
```

```
child = Child()
child.parent_method()
child.child_method()
```

Diagram:

```
Parent
↓
Child
```

2. Multilevel Inheritance

Chain of inheritance - child becomes parent for next class.

```
class GrandParent:
def gp_method(self):
print("GrandParent method")
```

```
class Parent(GrandParent):
def p_method(self):
print("Parent method")
```

```
class Child(Parent):
def c_method(self):
print("Child method")
```

```
child = Child()
child.gp_method() # From GrandParent
child.p_method() # From Parent
child.c_method() # Own method
```

Diagram:

```
GrandParent
↓
Parent
↓
Child
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

3. Hierarchical Inheritance

Multiple child classes inherit from one parent class.

```
class Animal:
    def eat(self):
        print("Eating")

class Dog(Animal):
    def bark(self):
        print("Barking")

class Cat(Animal):
    def meow(self):
        print("Meowing")

class Cow(Animal):
    def moo(self):
        print("Mooing")

dog = Dog()
dog.eat() # From Animal
dog.bark() # Own method
```

Diagram:

```
Animal
/ |
Dog Cat Cow
```

4. Multiple Inheritance

One child class inherits from multiple parent classes.

```
class Father:
    def father_trait(self):
        print("Father's trait")

class Mother:
    def mother_trait(self):
        print("Mother's trait")

class Child(Father, Mother):
    def child_trait(self):
        print("Child's trait")

child = Child()
child.father_trait() # From Father
child.mother_trait() # From Mother
child.child_trait() # Own method
```

Diagram:

```
Father Mother
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

\\
Child

5. Hybrid Inheritance

Combination of two or more types of inheritance.

```
class A:
def method_a(self):
print("Method A")

class B(A):
def method_b(self):
print("Method B")

class C(A):
def method_c(self):
print("Method C")

class D(B, C):
def method_d(self):
print("Method D")

d = D()
d.method_a() # From A
d.method_b() # From B
d.method_c() # From C
d.method_d() # Own method
```

Diagram:

A
/
B C
\\
D

14. Polymorphism

What is Polymorphism?

"Many forms" - Same interface with different implementations.

Types:

- Method Overriding (Runtime Polymorphism)
 - Method Overloading (Compile-time Polymorphism)
 - Operator Overloading
-

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajnagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

15. Method Overriding (Polymorphism)

Same method name in parent and child with different implementation.

Example 1: Basic Overriding

```
class Shape:
    def area(self):
        print("Calculate area")

class Rectangle(Shape):
    def __init__(self, length, width):
        self.length = length
        self.width = width
```

```
    def area(self): # Override
        return self.length * self.width
```

```
class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius
```

```
    def area(self): # Override
        return 3.14 * self.radius ** 2
```

```
rect = Rectangle(10, 5)
print(rect.area()) # 50
```

```
circle = Circle(7)
print(circle.area()) # 153.86
```

Example 2: Polymorphism in Action

```
class Animal:
    def speak(self):
        pass

class Dog(Animal):
    def speak(self):
        return "Woof!"

class Cat(Animal):
    def speak(self):
        return "Meow!"

class Cow(Animal):
    def speak(self):
        return "Moo!"
```

OUR LOCATIONS

Polymorphism

```
animals = [Dog(), Cat(), Cow()]
```

```
for animal in animals:  
    print(animal.speak())
```

Output: Woof!

Meow!

Moo!

16. Method Overloading (Polymorphism)

Python doesn't support traditional method overloading, but we can achieve it using default arguments.

Example 1: Using Default Arguments

```
class Calculator:  
    # Method overloading using default arguments  
    def add(self, a, b=0, c=0):  
        return a + b + c  
  
calc = Calculator()  
print(calc.add(10)) # 10  
print(calc.add(10, 20)) # 30  
print(calc.add(10, 20, 30)) # 60
```

*Example 2: Using args

```
class Math:  
    def add(self, *args):  
        return sum(args)  
  
math = Math()  
print(math.add(10)) # 10  
print(math.add(10, 20)) # 30  
print(math.add(10, 20, 30, 40)) # 100
```

Example 3: Type-based Overloading

```
class Display:  
    def show(self, data):  
        if isinstance(data, str):  
            print(f"String: {data}")  
        elif isinstance(data, int):  
            print(f"Integer: {data}")
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

```
elif isinstance(data, list):
    print(f"List: {data}")

display = Display()
display.show("Hello") # String: Hello
display.show(100) # Integer: 100
display.show([1, 2, 3]) # List: [1, 2, 3]
```

17. Operator Overloading (Polymorphism)

Using magic methods to overload operators.

Example 1: Overload + Operator

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Point(self.x + other.x, self.y + other.y)

    def __str__(self):
        return f"({self.x}, {self.y})"
```

```
p1 = Point(2, 3)
p2 = Point(4, 5)
p3 = p1 + p2 # Calls add()
print(p3) # (6, 8)
```

Example 2: Multiple Operator Overloading

```
class Number:
    def __init__(self, value):
        self.value = value

    def __add__(self, other):
        return Number(self.value + other.value)

    def __sub__(self, other):
        return Number(self.value - other.value)

    def __mul__(self, other):
        return Number(self.value * other.value)
```

OUR LOCATIONS

```
def __str__(self):  
    return str(self.value)
```

```
num1 = Number(10)  
num2 = Number(5)  
  
print(num1 + num2) # 15  
print(num1 - num2) # 5  
print(num1 * num2) # 50
```

Example 3: Comparison Operator Overloading

```
class Student:  
    def __init__(self, name, marks):  
        self.name = name  
        self.marks = marks
```

```
    def __eq__(self, other):  
        return self.marks == other.marks  
  
    def __lt__(self, other):  
        return self.marks < other.marks  
  
    def __gt__(self, other):  
        return self.marks > other.marks
```

```
s1 = Student("Rahul", 85)  
s2 = Student("Priya", 90)  
  
print(s1 == s2) # False  
print(s1 < s2) # True  
print(s1 > s2) # False
```

Practice Exercises

Exercise 1: Aggregation

Create a Library class that has multiple Book objects. Implement add and display methods.

Exercise 2: Single Inheritance

Create Vehicle parent class and Car child class with attributes and methods.

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Exercise 3: Multilevel Inheritance

Create Person → Employee → Manager hierarchy with appropriate attributes.

Exercise 4: Multiple Inheritance

Create a class that inherits from two parent classes with different attributes.

Exercise 5: Method Overriding

Create animal hierarchy where each animal overrides speak() method differently.

Exercise 6: super() Practice

Use super() to call parent constructor and methods in multilevel inheritance.

Exercise 7: Operator Overloading

Create a Vector class and overload +, -, and * operators.

Exercise 8: Polymorphism

Create a list of different shape objects and calculate area using polymorphism.

Quick Reference

Concept	Syntax
Inheritance	class Child(Parent):
super()	super().__init__()
Method Override	Same method name in child
Multiple Inherit	class Child(P1, P2):
Operator Overload	def __add__(self, other):

Table 2: OOP Part 3 Quick Reference

Key Concepts Summary

Inheritance Benefits:

- Code reusability
- Establishes IS-A relationship
- Supports polymorphism
- Easy to extend functionality

Polymorphism Types:

- Method Overriding - Different implementation in child
- Method Overloading - Same method, different parameters
- Operator Overloading - Custom operator behavior

OUR LOCATIONS

super() Uses:

- Call parent constructor
- Call parent methods
- Maintain inheritance chain
- Avoid code duplication

End of Notes

*Prepared by: Bikkad IT Institute
For: Python Beginner Level Students
Date: February 2026*

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune